

SentinelSoC - Top level Documentation

1. Overview

SentinelSoC is a single-clock-domain RISC-V System-on-Chip designed for secure firmware authentication in IoT devices. Its primary security property is that firmware is cryptographically verified by a dedicated hardware accelerator before the processor is permitted to execute it.

Only after Ed25519 signature verification passes does the bootrom unlock ISRAM for execution.

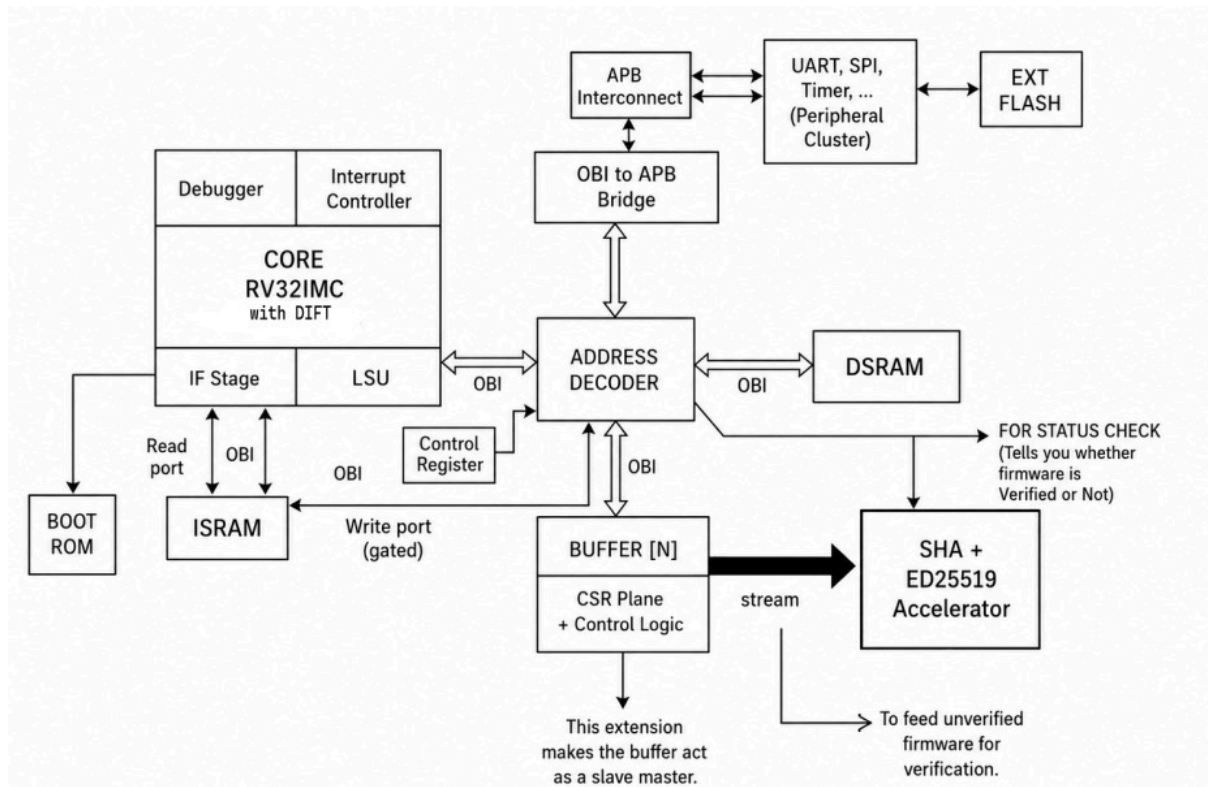
The design is built around:

- lowRISC Ibex RV32IMC core
- OBI (Open Bus Interface) interconnect
- OBI-to-APB bridge
- SHA-512 + Ed25519 verification pipeline

1.1 Key Features

- RV32IMC core (Ibex) with writeback stage
- Hardware-enforced secure boot
- ISRAM write-once lock after authentication
- OBI interconnect with 8-slave data demux
- OBI-to-APB bridge
- 12-source PLIC interrupt controller
- Standard RISC-V debug infrastructure
- Debug access gating support
- Synthesized on Artix-7 FPGA

2. System Architecture



The SoC uses a hub-and-spoke topology.

The core exposes:

- Instruction Fetch OBI master
- Data Access OBI master

Both connect to the Address Decoder.

The Address Decoder routes requests to:

- BootROM
- ISRAM
- DSRAM
- Control Registers
- Buffer CSR
- SHA/ED25519 CSR
- APB bridge
- Debug module

The fetch path only accesses:

- BootROM
- ISRAM

3. Memory Map

Peripheral	Base Address	End Address	Size	Access
BootROM	0x0000_0000	0x0000_0FFF	4 KB	Read-only
ISRAM	0x0001_0000	0x0001_0FFF	4 KB	R/W gated
DSRAM	0x0002_0000	0x0002_0FFF	4 KB	R/W
Control Regs	0x0003_0000	0x0003_0FFF	4 KB	R/W
Buffer CSR	0x0004_0000	0x0004_0FFF	4 KB	R/W
SHA/ED25519 CSR	0x0005_0000	0x0005_0FFF	4 KB	R/W
APB Region	0x1000_0000	0x1FFF_FFFF	256 MB	R/W
Debug Module	0x1A11_0000	0x1A11_FFFF	64 KB	R/W

3.1 APB Peripheral Map

Peripheral	Base Address	End Address
QSPI Flash Controller	0x1050_0000	0x1050_0FFF
Timer	0x1050_1000	0x1050_1FFF
SPI	0x1050_2000	0x1050_2FFF
UART	0x1050_3000	0x1050_3FFF
GPIO	0x1060_0000	0x1060_0FFF

4. Processor Core - RV32IMC with DIFT

Configuration

- PMPEnable = 0
- RV32M = RV32MFast
- WritebackStage = 1
- ICache = 0
- DbgTriggerEn = 0

Boot address:

0x0000_0000

Interrupts:

- irq_external from PLIC
- irq_timer tied to 0
- irq_software tied to 0

5. Interconnect - soc_addr_decoder

The interconnect wraps two obi_demux instances.

Fetch Path

Routes only to:

- BootROM
- ISRAM

Data Path Priority

1. BootROM
2. ISRAM
3. DSRAM
4. Control Registers
5. Buffer CSR
6. SHA/ED25519 CSR
7. Debug Module
8. APB Bridge
9. Error Responder

6. Control Register Block

Base Address:

0x0003_0000

Register Map

Offset	Name	Type	Description
0x00	CTRL0	RW	isram_lock
0x04	STATUS0	RO	crypto_verified
0x08	BOOT_STATUS	RO	boot_done
0x0C	CTRL1	RW	boot_done_set

ISRAM Lock

CTRL0[0]

Once set:

- ISRAM becomes read-only
- Only reset clears it
- Hardware-enforced integrity protection

7. Firmware Streaming Buffer - soc_buffer

Base Address:

0x0004_0000

Buffer FSM States

- IDLE
- FILLING
- STREAMING
- CTRL_SEND
- WAITING
- CAPTURING
- ERROR

Important CSRs

Offset	Register	Purpose
0x000	BUF_CTRL	control
0x004	BUF_STATUS	status
0x008	BUF_FILL_LVL	fill level
0x00C	BUF_BLK_SIZE	block size
0x010	IRQ_ENABLE	interrupt enables
0x014	IRQ_STATUS	interrupt status
0x018	ACCEL_CTRL	accelerator flags
0x01C	MSG_LENGTH	message length

8. SHA-512 + Ed25519 Accelerator

This is the hardware root of trust.

Implements:

- SHA-512 hashing
- Ed25519 verification
- Barrett reduction
- Point arithmetic
- Shared iterative multiplier
- Microsequencer-driven datapath

Verification Flow

Given:

Public key A

Message M

Signature (R, S)

Steps:

Compute:

$h = \text{SHA-512}(R \parallel A \parallel M) \bmod I$

Compute:

$P1 = S \cdot G$

Compute:

$P2 = R + h \cdot A$

Verify:

$8 \cdot P1 == 8 \cdot P2$

If equal:

signature_valid = 1

9. Debug Infrastructure

Uses standard RISC-V debug architecture.

Components:

- dmi_jtag
- dm_top

Debug module mapped at:

0x1A11_0000

Supports:

- halt
- resume
- abstract commands

10. Interrupt Architecture

The core exposes three dedicated level-sensitive interrupt input pins: `irq_software_i`, `irq_timer_i`, and `irq_external_i`. These correspond to machine software, timer, and external interrupts respectively as defined in the RISC-V Privileged Specification. The pins are driven by two SoC-level peripheral blocks, the CLINT and the PLIC, which are described in the following subsections.

CLINT - Core Local Interruptor :

The CLINT is a memory-mapped peripheral responsible for generating machine-mode timer and software interrupts. It drives the core's `irq_timer_i` and `irq_software_i` pins directly.

- **Machine Timer Interrupt (`irq_timer_i`):** The CLINT contains a 64-bit real-time counter (`mtime`) and a per-hart compare register (`mtimecmp`). A timer interrupt is asserted when `mtime` is greater than or equal to `mtimecmp` and stays level-high until software updates `mtimecmp` to a future value. This is the standard mechanism used for OS scheduling ticks and software timeouts.
- **Machine Software Interrupt (`irq_software_i`):** A software interrupt is triggered by writing to the memory-mapped `msip` register in the CLINT. This is typically used for self-interruption or for cross-hart signaling in multi-core configurations.

PLIC - Platform Level Interrupt Controller :

The PLIC is a memory-mapped peripheral that collects interrupt requests from external SoC peripherals and presents a single aggregated interrupt to the core's `irq_external_i` pin. It implements the RISC-V PLIC specification with priority-based arbitration and per-source enable masking. The general flow of operation is as follows:

- Each interrupt source is assigned a programmable priority level by software.
- A global priority threshold register allows the core to mask all sources at or below the set threshold.
- The PLIC asserts `irq_external_i` when any enabled source with priority above the threshold is pending.
- The core reads the PLIC claim register to identify and atomically acknowledge the highest-priority pending source.
- After the interrupt has been serviced, the core writes the source ID back to the PLIC completion register to re-arm it.

- 12 interrupt sources
- 1 target

Connected peripherals:

Source	Peripheral
irq_uart	UART
irq_spi	SPI
irq_qspi	QSPI
irq_gpio	GPIO
irq_timer_periph	Timer
irq_buf	Buffer
irq_sha	SHA Accelerator

11. Secure Boot Flow

Boot Sequence

1. Ibex starts at BootROM
2. BootROM streams firmware into Buffer
3. Buffer feeds SHA/Ed25519 accelerator
4. Accelerator verifies firmware
5. If valid:
 - ISRAM lock enabled
 - firmware copied to ISRAM
6. CPU jumps to:

0x0001_0000
7. Firmware executes from authenticated ISRAM

12. OBI-to-APB Bridge

Converts OBI transactions into APB3.

Connected peripherals:

- UART
- SPI
- QSPI
- GPIO
- Timer

14. References

- RFC 8032 EdDSA
- lowRISC Ibex
- OpenHW OBI Spec
- RISC-V Debug Spec